
Pigeonhole protocol design

Threebit Hacker
David Stainton

Abstract

In this specification we describe the components and protocols that compose Pigeonhole scattered storage. We define the behavior of communication clients that send and retrieve individual messages, BACAP streams, and AllOrNothing streams. Client actions are mediated through courier services that interact with storage replicas.

Introduction

Pigeonhole scattered storage enables persistent anonymous communication in which participants experience a coherent sequence of messages or a continuous data stream, but where user relationships and relations between data blocks are unlinkable not only from the perspective of third-party observers, but also from that of the mixnet components. This latter attribute provides resilience against deanonymization by compromised mixnet nodes.

The data blocks that Pigeonhole stores are supplied by the BACAP (Blinding-and-Capability) scheme. The Pigeonhole protocol scatters messages around many storage servers and among a space of BACAP box IDs. From a passive network observer's perspective, all of this is seemingly random. All communication among users consists of user-generated read or write queries to Pigeonhole storage, never directly to other users.

Many derivative protocols can be composed using Pigeonhole communication channels, including group communications. For details about the protocol specified here, see section "§5.6. End-to-end reliable group channels" in our paper Echomix: a Strong Anonymity System with Messaging [<https://arxiv.org/abs/2501.02933>]. For more information about BACAP, see §4 of the paper. For an understanding of how the core BACAP primitives are implemented, see `bacap.go` [<https://github.com/katzenpost/hpqc/blob/main/bacap/bacap.go>].

Message-layout snippets in this specification are given in the `trunnel` binary-format description language (matching `pigeonhole/pigeonhole_messages.trunnel`); code snippets showing in-memory courier or replica state are given in Go.

Glossary

- **Box:** BACAP's unit of data storage. Each box has a box ID (which also serves as its public key), a signature, and a ciphertext signed payload.
- **Courier:** Service that runs on a service node and interacts with storage replicas. Proxies requests from clients and routes replies back to clients (via SURBs).
- **Storage replica:** The actual storage nodes where message ciphertexts are stored and retrieved.
- **Intermediate replica:** "Intermediate replicas are chosen independently of the two final replicas for that box ID which are derived using the sharding scheme. The reason Alice designates intermediate replicas, as opposed to addressing the final replicas directly, is to avoid revealing to the courier which shard the box falls into." (From "§5.4.1. Writing messages" in the paper [<https://arxiv.org/abs/2501.02933>].) **Designated replica** (a.k.a. *final replica*, *shard replica*): One of the two replicas selected deterministically for a given box ID by the `Shard2` consistent-hash algorithm. The intermediate replicas replicate writes through to the designated replicas.
- **EnvelopeHash:** `BLAKE2b-256(CourierEnvelope.sender_pubkey || CourierEnvelope.ciphertext)`. Used by the courier for deduplication of retransmissions and for demultiplexing replica replies.

- **MKEM:** Multi-recipient KEM addressed to the pair of intermediate replicas. One MKEM ciphertext carries, for each recipient, a separate DEK encapsulation (**Dek1**, **Dek2**); either recipient may decapsulate and recover the padded **ReplicaInnerMessage**.
- **Replica-epoch:** A one-week period, distinct from the 20-minute mixnet PKI epoch, during which a given replica-side MKEM envelope keypair is valid. See the section called “Epochs” below.

Storage deployment requirements

A conforming deployment **MUST** run at least **four** storage replicas ($n \geq 4$). This enables the intermediate-replica unlinkability property described in the section called “Glossary” and in §5.4.1 of the paper: for each **CourierEnvelope**, the client picks two *intermediate* replicas which **MUST** be disjoint from the two *final* (sharded) replicas that hold the box. Disjointness is what prevents the courier from learning which shard the box falls into.

A conformant deployment and its undesirable alternatives can be compared as follows.

- **$n \geq 4$:** At least two non-shard replicas exist, so the intermediate set can be drawn uniformly at random from the non-shard subset, and disjointness is preserved.
- **$n = 3$:** Only one replica sits outside any given two-shard set, so the client cannot pick two independent non-shard intermediates. Implementations in this case fall back to an intermediate set that includes at least one shard replica, which tells the courier that replica is one of the two shards for the box.

The resulting information exposure is bounded:

- The courier does **NOT** learn the box ID. The box ID lives inside the MKEM-encrypted inner message addressed to the intermediate replicas; only the replicas can decrypt it.
- The courier does **NOT** learn the other shard member. Knowing one element of the two-element shard set does not constrain the identity of the other among the remaining replicas.
- The courier does **NOT** gain a way to link boxes in the same sequence. BACAP’s unlinkability of consecutive box IDs (paper §4) is a property of the capability-derivation scheme and is independent of sharding.
- **$n = 2$:** There is only one possible shard set, and the intermediate set is necessarily identical to it.

Consequently, the main unlinkability properties of Pigeonhole are not lost at $n = 3$, but the defense-in-depth margin provided by disjoint intermediate replicas is. Operators **SHOULD** treat $n \neq 4$ as the minimum supported configuration.

Epochs

Katzenpost has two distinct notions of “epoch” that operate on very different timescales, and Pigeonhole touches both:

- **Mixnet epoch** (a.k.a. *normal epoch*, *PKI epoch*): the short cadence on which the directory authorities publish a new PKI document. The default is 20 minutes. Mix nodes, gateways, service nodes, and clients all synchronise on this epoch. Sphinx replay keys on mix nodes rotate once per mixnet epoch.
- **Pigeonhole storage-replica epoch:** the long cadence on which each storage replica rotates its MKEM envelope keypair. The default is one week. A replica publishes, in every mixnet-epoch PKI descriptor, two envelope public keys: the current replica-epoch key and the next replica-epoch key.

Epoch tolerance for CourierEnvelope

A **CourierEnvelope** carries an **epoch** field that identifies the replica epoch whose envelope key the client used to encrypt the MKEM ciphertext. Conforming couriers and storage replicas **MUST**

accept epoch # $\{\text{current} - 1, \text{current}, \text{current} + 1\}$ where `current` is the courier's / replica's own view of the current replica epoch.

The `current - 1` tolerance handles the grace window immediately after a replica-epoch boundary, when a client with a slightly stale PKI view still encrypts to the previous envelope public key. Combined with `current`, this gives a **two replica-epoch data TTL** — roughly two weeks — because the future replica-epoch key is by definition a key that hasn't started being used yet, so `current` and `current - 1` are the epochs where actual data flows.

The `current + 1` tolerance handles the same boundary seen from the other side: a client whose clock or PKI view is slightly *ahead* of the courier / replica.

Envelopes outside this three-epoch window **MUST** be rejected, because by definition no replica still holds the matching decapsulation key:

- Older than `current - 1`: the envelope public key has been pruned from replicas (see the replica's envelope-key GC worker). The replica cannot decrypt and the courier can fail fast rather than forward a doomed request.
- Newer than `current + 1`: no replica has generated that envelope key yet (replicas generate `current` and `current + 1` only).

Couriers **SHOULD** reject with a dedicated envelope-level error code (`EnvelopeErrorInvalidEpoch`) so clients can distinguish "stale encryption" from other courier-side rejections.

Pigeonhole message format and constants

The Pigeonhole message types are defined in trunnel at `pigeonhole/pigeonhole_messages.trunnel` [https://github.com/katzenpost/katzenpost/blob/main/pigeonhole/pigeonhole_messages.trunnel]; the Go bindings live in `pigeonhole/trunnel_messages.go` [https://github.com/katzenpost/katzenpost/blob/main/pigeonhole/trunnel_messages.go]. All integer fields are big-endian and all variable-length fields carry an explicit length prefix. This trunnel encoding replaces the earlier CBOR encoding with a fixed-overhead binary format whose serialized size can be computed deterministically from the Sphinx geometry.

Carriage of these messages differs by hop:

- **Client** → **Courier**: a `CourierQuery` is carried inside a Sphinx packet payload. (See the section called “`CourierQuery`” for details.) The reverse direction uses a SURB supplied by the client.
- **Courier** → **Replica** and **Replica** → **Replica**: the courier and replicas do not use Sphinx packets between themselves. They communicate over the Katzenpost wire protocol defined in `core/wire` and `core/wire/commands`; the relevant commands are `ReplicaMessage`, `ReplicaMessageReply`, `ReplicaWrite`, and `ReplicaWriteReply`. Some of these commands embed trunnel-serialized pigeonhole blobs as their payload.

Fundamental size constants

Table 1.

Constant	Value	Source
BACAP_BOX_ID_SIZE	32 bytes	Ed25519 public key
BACAP_SIGNATURE_SIZE	64 bytes	Ed25519 signature
HASH_SIZE	32 bytes	BLAKE2b-256
MKEM_DEK_SIZE	60 bytes	mkem.DEKSize
CTIDH-1024 public key	160 bytes	hpqc/nike/ctidh/ctidh1024
X25519 public key	32 bytes	hpqc/nike/x25519

Constant	Value	Source
Hybrid CTIDH-1024 $\times$ X25519 NIKE public key	192 bytes	sum of the above
BACAP payload encryption overhead	16 bytes	ChaCha20-Poly1305 AEAD
MKEM encapsulation overhead	28 bytes	ChaCha20-Poly1305 nonce (12) + tag (16)

Maximum BACAP payload

The maximum plaintext BACAP payload a single box can carry is derived *backwards* from the Sphinx `UserForwardPayloadLength` by subtracting every layer of framing and cryptographic overhead that sits between the Sphinx payload and the BACAP plaintext. The authoritative calculation is performed by `NewGeometryFromSphinx` in `pigeonhole/geo/geometry.go` [<https://github.com/katzenpost/katzenpost/blob/main/pigeonhole/geo/geometry.go>].

Informally:

```

MaxPlaintextPayloadLength
= UserForwardPayloadLength
- CourierQuery framing (1 byte: query_type discriminator)
- CourierEnvelope header (intermediate_replicas[2] + Dek1 + Dek2
                        + reply_index + epoch + sender_pubkey_len
                        + sender_pubkey + ciphertext_len)
- MKEM AEAD overhead (28 bytes)
- ReplicaInnerMessage framing (1 byte: message_type discriminator)
- ReplicaWrite header (box_id + signature + payload_len = 100 bytes)
- BACAP AEAD overhead (16 bytes)
- BACAP length prefix (4 bytes)

```

With a hybrid CTIDH-1024 $\times$ X25519 NIKE, the fixed per-packet overhead between `UserForwardPayloadLength` and the BACAP plaintext is on the order of 560 bytes; the exact figure depends on the configured NIKE scheme and should always be obtained from `geometry.NewGeometryFromSphinx()` rather than computed by hand.

The CourierEnvelope as seen by the courier

The courier, upon unwrapping the Sphinx payload of a client's packet, sees a `CourierQuery` whose content is a `CourierEnvelope` with the following layout:

```

struct courier_envelope {
    u8  intermediate_replicas[2];           // replica indices in the PKI
    u8  dek1[MKEM_DEK_SIZE];               // DEK encapsulation for replica 0
    u8  dek2[MKEM_DEK_SIZE];               // DEK encapsulation for replica 1
    u8  reply_index;                       // which replica's reply to prefer
    u64 epoch;                             // replica-epoch under which the
                                           // MKEM ciphertext was produced

    u16 sender_pubkey_len;
    u8  sender_pubkey[sender_pubkey_len]; // client's ephemeral hybrid NIKE pk
    u32 ciphertext_len;
    u8  ciphertext[ciphertext_len];        // MKEM-encrypted ReplicaInnerMessage
}

```

Notable points:

- The `ciphertext` is opaque to the courier. It is an MKEM envelope addressed to the *pair* of intermediate replicas; either replica can decapsulate using its own DEK (`dek1` or `dek2` respectively).

- The epoch field names the *replica-epoch* whose envelope keys were used to produce the MKEM ciphertext. See the section called “Epochs” for the tolerance window.
- Prior to encryption, the inner `ReplicaInnerMessage` is zero-padded to `ReplicaInnerMessageWriteSize()` so that reads, writes and tombstones produce MKEM ciphertexts of identical length.

The ReplicaInnerMessage as seen by a replica

Once an intermediate replica decrypts the MKEM envelope, it obtains a `ReplicaInnerMessage` — a discriminated union over `message_type`:

```
struct replica_inner_message {
    u8 message_type IN [0, 1];    // 0 = read, 1 = write
    union content[message_type] {
        0: struct replica_read  read_msg;    // { box_id[32] }
        1: struct replica_write write_msg;    // { box_id[32], signature[64],
                                                //   payload_len[u32], payload[] }
    };
}
```

A `ReplicaWrite` with `payload_len == 0` is a tombstone.

Message types and interactions

Overview

- A client sends a `CourierQuery` inside a Sphinx packet payload. The courier's reply travels back to the client by means of a SURB the client also supplied.
- A client always designates **two** intermediate replicas per `CourierEnvelope`. The courier dispatches the corresponding pair of `ReplicaMessage` wire commands, one to each intermediate, and collects up to two `ReplicaMessageReply` results.
- The `reply_index` field is a *preference* indicating which of the two replica replies the client would like forwarded first. It is not a strict selector: should the preferred slot still be empty when the courier is ready to respond, the courier falls back to whichever reply it does hold, and indicates in the `CourierEnvelopeReply` the actual index that was served (see `courier/server/plugin.go` [<https://github.com/katzenpost/katzenpost/blob/main/courier/server/plugin.go>]).
- Clients **MUST** resend identical `CourierEnvelope` bodies — same `sender_pubkey` and `ciphertext` — until they receive a reply. The courier deduplicates resends by `EnvelopeHash`. Only the Sphinx-layer SURB is rotated between retransmissions.

CourierQuery

A `CourierQuery` is the top-level discriminated union that a client places into a Sphinx packet payload for the courier:

```
struct courier_query {
    u8 query_type IN [0, 1];
    union content[query_type] {
        0: struct courier_envelope envelope;    // read or write a single box
        1: struct copy_command   copy_command; // AllOrNothing copy
    };
}
```

The symmetric reply is a `CourierQueryReply`:

```
struct courier_query_reply {
    u8 reply_type IN [0, 1];
```

```

    union content[reply_type] {
        0: struct courier_envelope_reply envelope_reply;
        1: struct copy_command_reply    copy_command_reply;
    };
}

```

CourierEnvelope

The `CourierEnvelope` layout was given in the section called “The `CourierEnvelope` as seen by the courier”. The client constructs one `CourierEnvelope` per read or write. The courier performs the following actions.

1. Computes `EnvelopeHash = BLAKE2b-256(sender_pubkey || ciphertext)`.
2. Verifies epoch # `{current - 1, current, current + 1}` per the replica-epoch tolerance window, otherwise rejecting with `EnvelopeErrorInvalidEpoch`.
3. Looks `EnvelopeHash` up in its dedup cache. If present, the courier returns the cached reply (or an ACK if no reply has yet arrived) without re-dispatching to replicas.
4. On a cache miss, the courier constructs two `ReplicaMessage` wire commands — one bound for `intermediate_replicas[0]` carrying `dek1`, the other for `intermediate_replicas[1]` carrying `dek2` — and forwards them over the wire protocol.
5. The courier immediately sends an ACK reply to the client so it may stop retransmitting.

ReplicaMessage (wire command)

`ReplicaMessage` is *not* a trunnel pigeonhole type; it is the `core/wire/commands` command sent from a courier to a replica. Its payload fields are copied verbatim from the matching `CourierEnvelope`:

```

// core/wire/commands
type ReplicaMessage struct {
    SenderEPubKey []byte // copied from CourierEnvelope.sender_pubkey
    DEK           *[MKEM_DEK_SIZE]byte // dek1 or dek2, depending on destination
    Ciphertext    []byte // copied from CourierEnvelope.ciphertext
}

```

The recipient replica decapsulates the MKEM envelope using the replica-epoch key that corresponds to `CourierEnvelope.epoch` (trying each of the three keys in its tolerance window), yielding a padded `ReplicaInnerMessage`. The replica then dispatches on `message_type` to `ReplicaRead` or `ReplicaWrite` handling.

ReplicaMessageReply (wire command)

In response to a `ReplicaMessage`, the courier expects an asynchronous `ReplicaMessageReply` wire command from the replica:

```

// core/wire/commands
type ReplicaMessageReply struct {
    ErrorCode      uint8 // see the replica error-code table below
    EnvelopeHash   *[HASH_SIZE]byte // lets the courier demultiplex the reply
    EnvelopeReply  []byte // MKEM-encrypted ReplicaMessageReplyInner
}

```

The `EnvelopeReply` byte blob is produced by the replica via the MKEM scheme's `EnvelopeReply()` method (see `replica/handlers.go` [<https://github.com/katzenpost/katzenpost/blob/main/replica/handlers.go>]). It carries a `ReplicaMessageReplyInnerMessage` — a discriminated union over either a `ReplicaReadReply` or a `ReplicaWriteReply` — padded to `ReplicaReplyInnerMessageReadSize()` so that read replies and write replies are indis-

tinguishable in size, and encrypted to the client's ephemeral NIKE public key under the replica's envelope keypair.

CourierBookKeeping

For each outstanding `EnvelopeHash`, the courier maintains an in-memory dedup entry. Its actual structure is:

```
// courier/server/plugin.go
type CourierBookKeeping struct {
    Epoch            uint64           // replica-epoch at cache insertion
    CreatedAt        time.Time       // for TTL eviction (~5 minutes)
    QueryType        uint8           // the query_type that produced this ent
    IntermediateReplicas [2]uint8
    EnvelopeReplies  [2]*commands.ReplicaMessageReply
}
```

Note that the courier does **not** cache SURBs or SURB timestamps. A client's SURB is consumed by the Sphinx-layer plugin infrastructure at the moment the courier emits a reply and is not retained by the courier's Pigeonhole state. Should the client not receive that reply, it is expected to retransmit a fresh Sphinx packet carrying a fresh SURB but the identical `CourierEnvelope` body; the courier, recognising the `EnvelopeHash`, replies using the new SURB.

The dedup cache has a TTL of 5 minutes (`DedupCacheTTL` in `courier/server/plugin.go` [<https://github.com/katzenpost/katzenpost/blob/main/courier/server/plugin.go>]).

CourierEnvelopeReply

The courier's reply to a `CourierEnvelope` has the following trunnel layout:

```
struct courier_envelope_reply {
    u8  envelope_hash[HASH_SIZE];           // identifies the originating envelope
    u8  reply_index;                        // which intermediate replica's reply
                                           // is being returned (may differ
                                           // from the client's requested index)
    u8  reply_type IN [0, 1];               // 0 = ACK, 1 = PAYLOAD
    u32 payload_len;
    u8  payload[payload_len];              // the MKEM-encrypted EnvelopeReply,
                                           // present iff reply_type == PAYLOAD
    u8  error_code;                        // see envelope error codes below
}
```

A `reply_type` of ACK (0) indicates the courier has received the envelope and dispatched it to the replicas but has not yet received a reply for the requested index. A `reply_type` of PAYLOAD (1) indicates the `payload` field carries the MKEM-encrypted `EnvelopeReply` produced by a replica.

Embedded pigeonhole types

These trunnel structs are not carried on the wire in isolation; they are embedded inside MKEM envelopes and their replies.

ReplicaRead

Embedded inside the MKEM-encrypted `ReplicaInnerMessage` a client sends to a replica (via the courier) for a read operation.

```
struct replica_read {
    u8 box_id[BACAP_BOX_ID_SIZE];
```

```
}
```

ReplicaReadReply

Embedded inside the MKEM-encrypted `ReplicaMessageReplyInnerMessage` a replica returns for a successful read. Padding is applied at the outer `ReplicaMessageReplyInnerMessage` level; this struct carries no padding of its own.

```
struct replica_read_reply {
    u8  error_code;
    u8  box_id[BACAP_BOX_ID_SIZE];
    u8  signature[BACAP_SIGNATURE_SIZE];
    u32 payload_len;
    u8  payload[payload_len];
}
```

ReplicaWrite

Used both (a) embedded inside the MKEM-encrypted `ReplicaInnerMessage` for a client write, and (b) carried directly as a `core/wire/commands` command between replicas during replication.

```
struct replica_write {
    u8  box_id[BACAP_BOX_ID_SIZE];
    u8  signature[BACAP_SIGNATURE_SIZE];
    u32 payload_len;
    u8  payload[payload_len];
}
```

A `ReplicaWrite` with `payload_len == 0` is a **tombstone**. Replicas treat tombstone writes as overwrites: an existing box at the same `box_id` is replaced by the tombstone, and subsequent reads return `ReplicaErrorTombstone`.

ReplicaWriteReply

Embedded inside a `ReplicaMessageReplyInnerMessage` on the client reply path, and also used as the `core/wire/commands` reply to inter-replica replication.

```
struct replica_write_reply {
    u8  error_code;
}
```

Tombstones

A `ReplicaWrite` whose `payload_len == 0` is a *tombstone*. It marks a box as deleted without revealing that fact to the courier.

- Replicas treat a tombstone write as an ordinary overwrite. An existing `Box` at the same `box_id` is replaced by the tombstone, and subsequent reads of that `box_id` return `ReplicaErrorTombstone`. (See the section called “Replica error codes”.)
- `ReplicaErrorTombstone` is an *expected* outcome rather than a failure; it positively confirms that the box was deleted, as distinct from `ReplicaErrorBoxNotFound`.
- Because the inner `ReplicaInnerMessage` is zero-padded to `ReplicaInnerMessageWriteSize()` before MKEM encryption (as described in the section called “The CourierEnvelope as seen by the courier”), a tombstone write produces an MKEM ciphertext of exactly the same length as a non-empty write or a read. A passive observer therefore cannot distinguish deletion from any other operation.

Tombstones are also used by the AllOrNothing copy protocol. After processing a `CopyCommand`, the courier overwrites every box of the temporary stream with tombstones.

EnvelopeHash

The `EnvelopeHash` uniquely identifies a `CourierEnvelope` for the purposes of deduplication and reply demultiplexing. It is computed as:

```
EnvelopeHash = BLAKE2b-256(sender_pubkey || ciphertext)
```

where `sender_pubkey` and `ciphertext` are the corresponding fields of the `CourierEnvelope`. The implementation is `CourierEnvelope.EnvelopeHash()` in `pigeonhole/helpers.go` [<https://github.com/katzenpost/katzenpost/blob/main/pigeonhole/helpers.go>].

A retransmitted `CourierEnvelope` carries the same `sender_pubkey` and `ciphertext` as the original, and therefore hashes to the same value; only the surrounding Sphinx packet (and its SURB) changes between attempts.

Error codes

All error codes are defined in `pigeonhole/errors.go` [<https://github.com/katzenpost/katzenpost/blob/main/pigeonhole/errors.go>].

Replica error codes

Returned by a replica in `ReplicaMessageReply.ErrorCode`, `ReplicaReadReply.error_code`, and `ReplicaWriteReply.error_code`.

Table 2.

Code	Name	Meaning
0	<code>ReplicaSuccess</code>	Operation completed successfully
1	<code>ReplicaErrorBoxIDNotFound</code>	Read miss (expected outcome)
2	<code>ReplicaErrorInvalidBoxID</code>	Malformed box ID
3	<code>ReplicaErrorInvalidSignature</code>	BACAP signature verification failed
4	<code>ReplicaErrorDatabaseFailure</code>	Transient RocksDB error
5	<code>ReplicaErrorInvalidPayload</code>	Malformed payload
6	<code>ReplicaErrorStorageFull</code>	Storage capacity exceeded
7	<code>ReplicaErrorInternalError</code>	Internal server error
8	<code>ReplicaErrorInvalidEpoch</code>	Replica-epoch envelope key unavailable
9	<code>ReplicaErrorReplicationFailed</code>	Replication to shard peer failed
10	<code>ReplicaErrorBoxAlreadyExists</code>	Idempotent-write outcome (expected)

Code	Name	Meaning
11	ReplicaErrorTombstone	Read returned a tombstone (expected)

Codes 1, 10 and 11 are "expected outcomes" — they correspond to normal protocol states rather than faults. The thin-client helper `thin.IsExpectedOutcome(err)` treats these three codes as non-errors.

Courier envelope error codes

Returned by the courier in `CourierEnvelopeReply.error_code`.

Table 3.

Code	Name	Meaning
0	EnvelopeErrorSuccess	Operation completed.
1	EnvelopeErrorInvalidEnvelope	Malformed envelope (e.g. <code>reply_index > 1</code>)
2	EnvelopeError-CacheCorruption	Internal cache inconsistency
3	EnvelopeErrorPropagationError	Failed to dispatch to replicas.
4	EnvelopeErrorInvalidEpoch	<code>CourierEnvelope.epoch</code> outside tolerance window.

Copy command status codes

Returned by the courier in `CopyCommandReply.status` (see below).

Returned by the courier in the `status` field of a `CopyCommandReply`. (See the section called “CopyCommandReply” for more information.)

Table 4.

Code	Name	Meaning
0	CopyStatusSucceeded	All destination writes completed.
1	CopyStatusInProgress	Courier has accepted the command; processing continues.
2	CopyStatusFailed	Aborted; see <code>error_code</code> + <code>failed_envelope_index</code> .

Sharding and replica selection

For each box, two *designated* (final) replicas are derived deterministically from the box ID using the `Shard2` consistent-hash algorithm. (See `replica/common/shard.go` [<https://github.com/katzenpost/katzenpost/blob/main/replica/common/shard.go>].):

```
for each online replica r with identity key k_r:
    h_r = BLAKE2b-256(k_r || box_id)
// return the two replicas whose h_r values are smallest
```

The two *intermediate* replicas chosen by the client for a given `CourierEnvelope` are drawn independently of the designated replicas, per `pigeonhole.GetRandomIntermediateReplicas`:

- **$n \geq 4$ replicas:** two intermediates are drawn uniformly at random from the replicas that are **not** in the designated (shard) set — preserving the disjointness invariant described in the section called “Storage deployment requirements”.
- **$n = 3$:** fallback — at least one intermediate must coincide with a designated replica; this weakens the intermediate/final disjointness guarantee but does not compromise box unlinkability.
- **$n < 3$:** rejected.

Intermediate replicas, upon accepting a `ReplicaWrite`, compute the designated replicas themselves and forward the `ReplicaWrite` to them via the `core/wire/commands/ReplicaWrite` command (see `replica/connector.go` [<https://github.com/katzenpost/katzenpost/blob/main/replica/connector.go>]).

Protocol sequence visualizations

For simplicity, the following diagrams omit replication while illustrating the Pigeonhole write and read operations.

Pigeonhole write operation

Figure 1. Annotated writes

Pigeonhole read operation

Figure 2. Annotated reads

Pigeonhole AllOrNothing protocol

The AllOrNothing delivery mechanism ensures that a set of associated BACAP writes either succeeds or fails atomically from the point of view of a replica or second-party client reader. This behavior prevents an adversary from detecting a correlation between (A) the sending client's failure to transmit multiple messages at once with (B) a network interruption on the sending client's side of the network. Regardless of the number of messages in the set, the adversary gets to observe "at most once" that the sending client interacted with the network.

The protocol works as follows.

Step 1

The client uploads a *temporary Pigeonhole stream*.

The stream is sequence of `CourierEnvelope`s. Each `CourierEnvelope` is serialized and prefixed with a single 4-byte (u32) length field giving the size of that one envelope; the resulting length-prefixed blobs are concatenated back-to-back into one continuous byte stream.

That byte stream is then split across the BACAP Boxes of the temporary stream. A serialized `CourierEnvelope` is strictly larger than the maximum BACAP box payload (it wraps a full box payload plus its own metadata), so a single envelope does not fit in one box and the concatenated stream necessarily spans several boxes. Envelope boundaries therefore do not align with box boundaries. The precise framing is given in the section called “Temporary stream data format”.

Step 2

The client sends a random courier the copy command which encapsulates the write capability to the temporary Pigeonhole stream written in Step 1 above. When the courier receives this copy command

it extracts the `ReadCap` from the given `WriteCap` and uses it to read the stream of data. The courier then reads a box at a time and tries to extract 0 or 1 envelopes from each accumulation of stream segments.

After processing the command, the courier then overwrites the temporary stream with tombstones.

Temporary stream data format

Each box in the temporary stream is a serialized `CopyStreamElement`. Defined in trunnel as:

```
// CopyStreamElement - wraps a CourierEnvelope chunk with stream position flags
// Overhead: 1 byte (flags) + 4 bytes (envelope_len) = 5 bytes
struct copy_stream_element {
    // Flags: bit 0 = isStart, bit 1 = isFinal
    u8 flags;

    // The CourierEnvelope serialized bytes
    u32 envelope_len;
    u8 envelope_data[envelope_len];
}
```

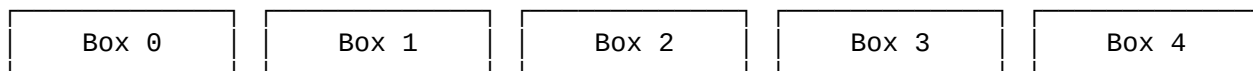
Here it is in golang:

```
type CopyStreamElement struct {
    Flags      uint8
    EnvelopeLen uint32
    EnvelopeData []uint8
}
```

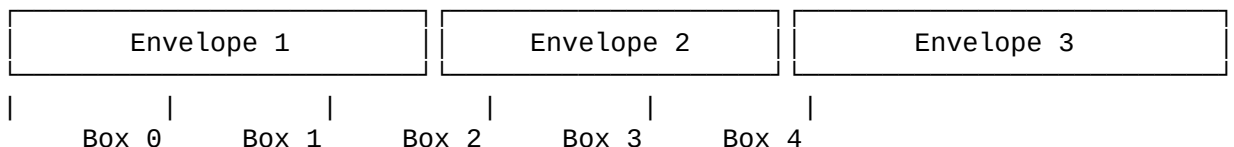
The purpose of this specific format is to use the `isStart` and `isFinal` flags to tell the courier the first box and last box of the stream to process. The payload encapsulated within the `EnvelopeData` fields of many of these `CopyStreamElements` is itself a stream of data which contains `CourierEnvelopes` with four-byte prefixes.

A key property of this encoding is that envelope boundaries do not align with box boundaries. Each BACAP box payload has a maximum size of N bytes, but a serialized `CourierEnvelope` (which contains a full box payload plus metadata) exceeds N bytes. Therefore envelopes are serialized into a continuous byte stream and split across multiple boxes in the temporary copy stream:

TEMPORARY COPY STREAM BOXES (each holds N bytes):



SERIALIZED ENVELOPE DATA (envelope boundaries don't align with box boundaries):



The courier reads the stream box by box, accumulating data until it can extract complete envelopes. The `isStart` and `isFinal` flags on the `CopyStreamElement` wrappers tell the courier where the stream begins and ends.

Each embedded `CourierEnvelope` is processed as a normal write and results in a `ReplicaMessage` being dispatched to the intermediate replicas over the wire protocol.

The courier does NOT need to keep track of the `EnvelopeHash` for each contained `CourierEnvelope` for the purpose of replying to the client (the "client" of these envelopes being, in this case,

the courier itself), but it does need to keep resending them until the intermediate replicas have ACKed them.

The courier **MUST** keep track of the hash of the `CopyCommand` (computed as `BLAKE2b-256(write_cap)`) and **MUST NOT** process a given command more than once. This dedup cache has a TTL of 30 minutes (`CopyDedupCacheTTL` in `courier/server/plugin.go` [<https://github.com/katzenpost/katzenpost/blob/main/courier/server/plugin.go>]).

CopyCommand

`CopyCommand` is sent by a client to its chosen courier after the client has successfully uploaded every box of the temporary stream. The trunnel layout is:

```
struct copy_command {
    u32 write_cap_len;
    u8  write_cap[write_cap_len];    // serialized BACAP BoxOwnerCap
}
```

On receipt, the courier:

1. Computes `copyKey = BLAKE2b-256(write_cap)`.
2. Consults its copy dedup cache. If an in-progress entry is found, the courier responds immediately with `CopyStatusInProgress`. If a completed entry within its TTL is found, the courier returns the cached terminal reply.
3. Otherwise, the courier reconstructs the `BACAPWriteCap` from the bytes, derives the corresponding `ReadCap`, and reads the temporary stream box by box, feeding the decrypted BACAP payloads into a `CopyStreamEnvelopeDecoder`.
4. Each complete `CourierEnvelope` emitted by the decoder is dispatched to its two intermediate replicas; the courier waits for acknowledgements with bounded retries and exponential backoff.
5. Upon processing the box bearing the `isFinal` flag (or upon an unrecoverable failure), the courier attempts — on a best-effort basis — to overwrite every box of the temporary stream with a tombstone.

Replica error handling during a `CopyCommand` is classified as **temporary** or **permanent** by `courier/server/copy_errors.go` [https://github.com/katzenpost/katzenpost/blob/main/courier/server/copy_errors.go]: transient errors (storage full, database failure, internal error, replication failed, box-ID not found) trigger bounded retries against the same shard before failover; permanent errors (invalid box ID, invalid signature, invalid payload, invalid epoch, box already exists, tombstone) cause immediate failover or abort.

CopyCommandReply

The courier's reply to a `CopyCommand` has the following trunnel layout:

```
struct copy_command_reply {
    u8  status;                // CopyStatus{Succeeded, InProgress, Failed}
    u8  error_code;            // replica error code (meaningful iff status != Succeeded)
    u64 failed_envelope_index; // 1-based sequential position in the
                                // CourierEnvelope stream at which processing
                                // stopped (meaningful iff status == Failed)
}
```

The status codes are enumerated in "Copy command status codes" above. `failed_envelope_index` counts *envelopes* within the stream, not boxes: the first envelope in the first box of the temporary stream is index 1.

A client receiving `CopyStatusInProgress` should continue polling — i.e. resend the same `CopyCommand` via a fresh SURB after a short interval (`CopyPollInterval`, currently 5 seconds)

— until it receives either `CopyStatusSucceeded` or `CopyStatusFailed`. Because the courier's copy dedup cache keys on `BLAKE2b-256(write_cap)`, these repeated polls will not cause the `CopyCommand` to be processed more than once.

Potential use cases of AllOrNothing

In no particular order:

- Atomically writing to two or more boxes.
 - The boxes can reside on distinct streams (or not); the courier doesn't know anything about streams of `CourierEnvelope`.
- Sending long messages that span more than one BACAP payload, like a file / document / picture.
- Group chat join uses AllOrNothing when adding a new member to the group:
 - The person introducing a new member writes to their group chat stream.
 - The person introducing a new member also writes to the existing conversation stream that the new member can already read.
- Group chat uses AllOrNothing in all cases where it needs to send long messages -- files, pictures, audio, long cryptographic keys such as the group membership list, and so on.

Protocol narration example

Alice wants to send a message to Bob, who is already connected to Alice. That is, Alice will write to a box with ID 12345 that Bob is trying to read from.

1. Alice sends:

A. SPHINX packet containing:

- SPHINX header
 - Routing commands to make it arrive at a courier (on a service provider)
- SPHINX payload
 - Reply SURB
 - `CourierEnvelope` encrypted for a courier chosen at random. The envelope tells the courier about two randomly chosen replicas ("intermediate replicas").

B. Alice doesn't know if the packet makes it through the network. Until she receives a reply, she keeps resending the `CourierEnvelope` to the same courier.

2. The courier receives the `CourierEnvelope` from Alice.

A. The courier records the `EnvelopeHash` of the `CourierEnvelope` in its `CourierBook-Keeping` datastructure.

B. If the courier has already seen that hash, GOTO step 4.

3. The courier sends an ACK to Alice using the SURB on file for Alice, telling her to stop resending the `CourierEnvelope`. If there's no SURB, the Courier waits for the next re-sent `CourierEnvelope` from Alice matching the `EnvelopeHash`.

A. The courier constructs two `ReplicaMessage` objects and puts them in its outgoing queue for the two intermediate replicas. (Note that each courier maintains constant-rate traffic with all replicas).

B. The courier keeps doing this for each intermediate replica until it receives an ACK from that replica.

Alice's actions are now complete.

4. The two intermediate replicas receive the two `ReplicaMessage` objects.
 - A. They decrypt them and compute the "designated replicas" based on the BACAP box ID.
 - B. The replicas put ACKs in their outgoing queues for the courier saying "we have received these messages" (see step 3.2), referencing the `EnvelopeHash`. This tells the courier to stop resending to the intermediate replicas.
 - C. They put the decrypted contents (a `ReplicaWrite` BACAP tuple: box ID 12345; signature; encrypted payload) in their outgoing queues for the "designated replicas".
 - D. Each replica waits for a reply from its designated replica.
 - E. When an intermediate replica receives its ACK from the designated replica, the intermediate replica has no more tasks.

Bob now wants to check if Alice has written a message at box 12345.

5. Similar to step 1, Bob sends a SPHINX packet to a courier chosen at random.
 - A. SPHINX packet containing:
 - SPHINX header
 - Routing commands to make it arrive at a courier (on a service provider).
 - SPHINX payload
 - The envelope tells the courier about two randomly chosen replicas ("intermediate replicas").
 - `CourierEnvelope` encrypted for a courier chosen at random. Its encrypted payload is a `ReplicaRead` command (reference box ID 12345).
 - B. Bob doesn't know if the packet makes it through the network. Until he receives a reply, he keeps resending the `CourierEnvelope` to the same courier.
6. The courier receives Bob's `CourierEnvelope`. (Similar to step 2).
7. The courier sends an ACK to Bob. (Similar to step 3.)
8. The two intermediate replicas receive the two `ReplicaMessage` objects containing Bob's `ReplicaRead`.
 - A. Similar to 4.A.
 - B. Similar to 4.B.
 - C. Similar to 4.C.
 - D. Similar to 4.D.
 - E. When the intermediate replica receives its ACK from the designated replica, it will include the (perhaps confusingly named) `ReplicaWrite` (BACAP tuple).
 - F. The intermediate replica wraps it in a `ReplicaMessageReply` encrypted for Bob's `ReplicaMessage.EPubKey` and puts it in its outgoing queue for the Courier.
9. The Courier receives two `ReplicaMessageReply` wire commands from the two intermediate replicas.

- A. It matches `ReplicaMessageReply.EnvelopeHash` to its recorded bookkeeping state (step 2.1).
 - B. It wraps the replica's `EnvelopeReply` blob (the MKEM-encrypted `ReplicaMessageReplyInnerMessage`) into a `CourierEnvelopeReply` with `reply_type = PAYLOAD`, and — if Bob's latest retransmission arrived with a fresh Sphinx SURB — forwards the resulting `CourierQueryReply` back through the mixnet to Bob.
 - C. Either way the courier retains the `ReplicaMessageReply` in its dedup cache for the cache TTL (5 minutes), so that a subsequently-arriving retransmission of the same envelope can be served from cache without re-dispatching to replicas.
10. Bob keeps resending his `CourierEnvelope` from step 5 until he receives a `CourierEnvelopeReply` with `reply_type = PAYLOAD`.
- A. Bob decapsulates the MKEM `EnvelopeReply` with the private key corresponding to the `sender_pubkey` he put in his `CourierEnvelope`, obtaining the padded `ReplicaMessageReplyInnerMessage`.
 - B. Once unpadded, the inner message is either a `ReplicaReadReply` carrying the BACAP tuple (box ID, signature, ciphertext) or a non-success `error_code`. If the code is `ReplicaErrorBoxIDNotFound`, Bob waits and polls again (i.e. returns to step 5). If it is `ReplicaErrorTombstone`, Alice has deleted the message. Otherwise Bob BACAP-decrypts and verifies the payload.

Bob can now read Alice's message.